Actividad_5_KNN_Wine_ejecutado

October 28, 2026

1 Actividad 5 — Clasificación supervisada con KNN

Maestría en Ciencia de Datos — Fundamentos de Inteligencia Artificial

1.1 Objetivo

Implementar y evaluar un clasificador K-Nearest Neighbors (KNN), justificando la selección del hiperparámetro K.

Dataset: Wine Recognition (UCI Machine Learning Repository, distribuido mediante scikit-learn).

- 178 observaciones.
- 13 características químicas.
- 3 clases.
- Clasificación supervisada multiclase.

Metodología: EDA, división estratificada, estandarización, validación cruzada, selección de K y evaluación final.

[1]:
```
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedKFold,␣
 ↪cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    accuracy_score, balanced_accuracy_score,
    precision_score, recall_score, f1_score,
    classification_report, ConfusionMatrixDisplay
)

ROOT = Path.cwd()
if not (ROOT / "data" / "wine.csv").exists():
    ROOT = ROOT.parent
```

```
DATA = ROOT / "data" / "wine.csv"
assert DATA.is_file(), f"No se encontro el dataset: {DATA}"

df = pd.read_csv(DATA)
print("Dataset cargado:", df.shape)
```

```
Dataset cargado: (178, 14)
```

1.2 1. Análisis exploratorio

Se estudia la calidad del conjunto de datos, la distribución de clases y la separación visual de las observaciones.

[2]:
```
print("Registros:", len(df))
print("Duplicados exactos:", df.duplicated().sum())
print("Valores faltantes:", df.isna().sum().sum())
print("\nDistribucion por clase:")
print(df["target"].value_counts().sort_index())
print("\nEstadisticas descriptivas:")
display(df.describe().round(3))
```

```
Registros: 178
Duplicados exactos: 0
Valores faltantes: 0

Distribucion por clase:
target
0    59
1    71
2    48
Name: count, dtype: int64

Estadisticas descriptivas:
```

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	\
count	178.000	178.000	178.000	178.000	178.000	
mean	13.001	2.336	2.367	19.495	99.742	
std	0.812	1.117	0.274	3.340	14.282	
min	11.030	0.740	1.360	10.600	70.000	
25%	12.362	1.602	2.210	17.200	88.000	
50%	13.050	1.865	2.360	19.500	98.000	
75%	13.678	3.082	2.558	21.500	107.000	
max	14.830	5.800	3.230	30.000	162.000	

	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	\
count	178.000	178.000	178.000	178.000	
mean	2.295	2.029	0.362	1.591	
std	0.626	0.999	0.124	0.572	
min	0.980	0.340	0.130	0.410	

```
25%            1.742       1.205                  0.270            1.250
50%            2.355       2.135                  0.340            1.555
75%            2.800       2.875                  0.438            1.950
max            3.880       5.080                  0.660            3.580

       color_intensity      hue  od280/od315_of_diluted_wines   proline  \
count          178.000  178.000                       178.000   178.000
mean             5.058    0.957                         2.612   746.893
std              2.318    0.229                         0.710   314.907
min              1.280    0.480                         1.270   278.000
25%              3.220    0.782                         1.938   500.500
50%              4.690    0.965                         2.780   673.500
75%              6.200    1.120                         3.170   985.000
max             13.000    1.710                         4.000  1680.000

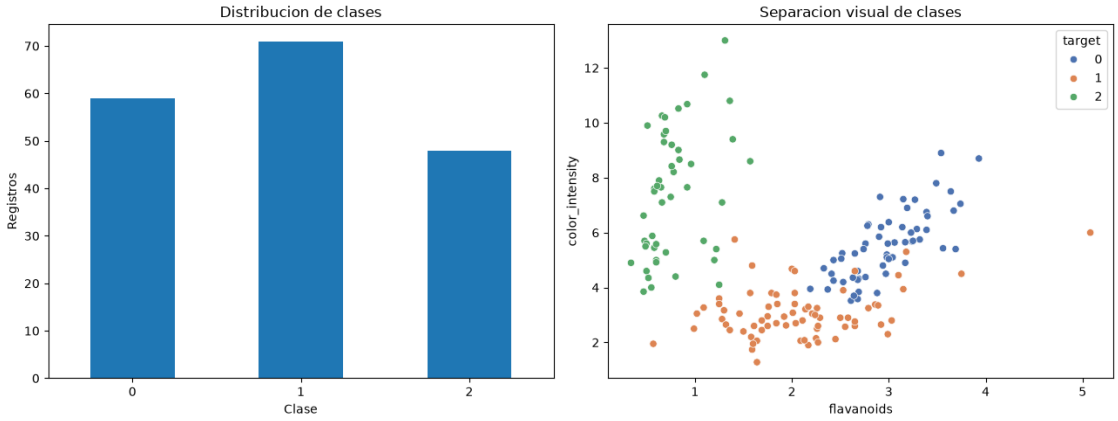
        target
count  178.000
mean     0.938
std      0.775
min      0.000
25%      0.000
50%      1.000
75%      2.000
max      2.000
```

[3]:
```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

df["target"].value_counts().sort_index().plot.bar(ax=axes[0])
axes[0].set_title("Distribucion de clases")
axes[0].set_xlabel("Clase")
axes[0].set_ylabel("Registros")
axes[0].tick_params(axis="x", rotation=0)

sns.scatterplot(
    data=df,
    x="flavanoids",
    y="color_intensity",
    hue="target",
    palette="deep",
    ax=axes[1]
)
axes[1].set_title("Separacion visual de clases")

plt.tight_layout()
plt.show()
```



1.3 2. Preparación de los datos

Se utiliza una división estratificada 80/20 con semilla 42.

El escalado se integra dentro de un Pipeline para garantizar que los parámetros de estandarización se calculen únicamente con los datos de entrenamiento de cada ajuste.

La prueba final permanecerá reservada durante la selección de K.

[4]:
```
X = df.drop(columns=["target"])
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    stratify=y,
    random_state=42
)

assert X_train.index.intersection(X_test.index).empty

print("Entrenamiento:", X_train.shape)
print("Prueba:", X_test.shape)
print("\nDistribucion de entrenamiento:")
print(y_train.value_counts().sort_index())
print("\nDistribucion de prueba:")
print(y_test.value_counts().sort_index())
```

```
Entrenamiento: (142, 13)
Prueba: (36, 13)

Distribucion de entrenamiento:
target
0    47
```

```
1    57
2    38
Name: count, dtype: int64

Distribucion de prueba:
target
0    12
1    14
2    10
Name: count, dtype: int64
```

1.4 3. Selección del mejor K

Se evalúan 60 valores de K mediante validación cruzada estratificada de cinco particiones.

Criterio: maximizar el accuracy promedio de validación.

En caso de empate numérico, seleccionar el menor K. Los datos de prueba no intervienen en esta selección.

[5]:
```
cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

resultados = []

for k in range(1, 61):
    pipeline = Pipeline([
        ("scaler", StandardScaler()),
        ("knn", KNeighborsClassifier(n_neighbors=k))
    ])

    scores = cross_val_score(
        pipeline, X_train, y_train,
        cv=cv, scoring="accuracy", n_jobs=1
    )

    resultados.append({
        "k": k,
        "accuracy_mean": float(scores.mean()),
        "accuracy_std": float(scores.std())
    })

tabla_k = pd.DataFrame(resultados)
maximo = tabla_k["accuracy_mean"].max()

candidatos = tabla_k[
```

```
    np.isclose(
        tabla_k["accuracy_mean"], maximo,
        rtol=0, atol=1e-12
    )
]

mejor_k = int(candidatos["k"].min())

print("Mejor K:", mejor_k)
print(f"Accuracy CV: {maximo:.4%}")
print("\nCinco mejores configuraciones:")
display(
    tabla_k.sort_values(
        ["accuracy_mean", "k"],
        ascending=[False, True]
    ).head(5)
)
```

```
Mejor K: 29
Accuracy CV: 97.8818%

Cinco mejores configuraciones:
```

	k	accuracy_mean	accuracy_std
28	29	0.978818	0.017301
29	30	0.971921	0.014050
30	31	0.971921	0.014050
32	33	0.971921	0.014050
25	26	0.971675	0.026665

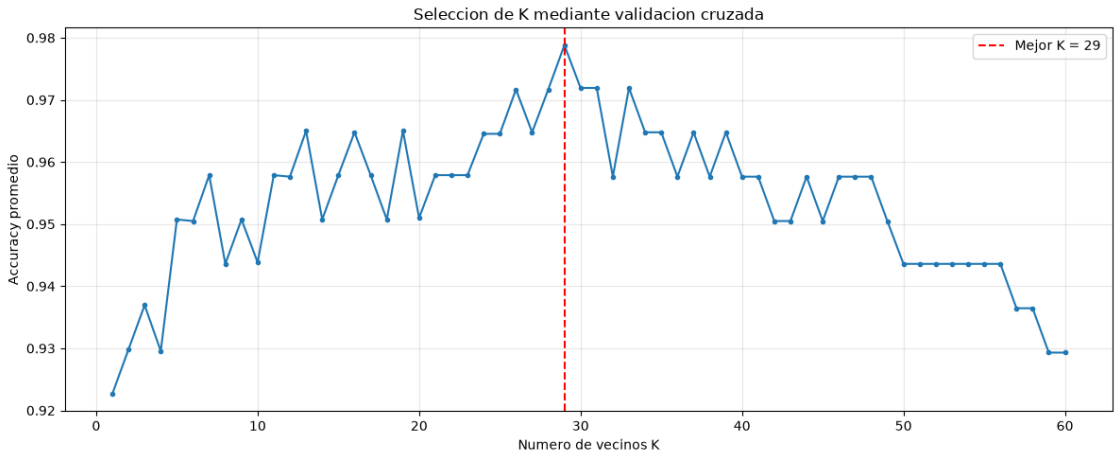
[6]:
```
fig, ax = plt.subplots(figsize=(12, 5))

ax.plot(
    tabla_k["k"],
    tabla_k["accuracy_mean"],
    marker="o",
    markersize=3
)
ax.axvline(
    mejor_k,
    color="red",
    linestyle="--",
    label=f"Mejor K = {mejor_k}"
)

ax.set_title("Seleccion de K mediante validacion cruzada")
ax.set_xlabel("Numero de vecinos K")
ax.set_ylabel("Accuracy promedio")
```

```
ax.grid(alpha=0.3)
ax.legend()

plt.tight_layout()
plt.show()
```



1.5 4. Evaluación final

Una vez seleccionado K, el modelo se ajusta utilizando todo el conjunto de entrenamiento.

Se evalúa sobre las 36 observaciones reservadas, utilizando accuracy, balanced accuracy, precision, recall, F1-score y una matriz de confusión.

El conjunto de prueba no se utiliza para reajustar K.

[7]:
```
modelo_final = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier(n_neighbors=mejor_k))
])

modelo_final.fit(X_train, y_train)
y_pred = modelo_final.predict(X_test)

metricas = {
    "accuracy": accuracy_score(y_test, y_pred),
    "balanced_accuracy": balanced_accuracy_score(y_test, y_pred),
    "precision_macro": precision_score(y_test, y_pred, average="macro"),
    "recall_macro": recall_score(y_test, y_pred, average="macro"),
    "f1_macro": f1_score(y_test, y_pred, average="macro")
}

print("METRICAS FINALES")
```

```
for nombre, valor in metricas.items():
    print(f"{nombre}: {valor:.4f}")

print("\nREPORTE DE CLASIFICACION")
print(classification_report(y_test, y_pred, digits=4))

print("Aciertos:", int((y_test.to_numpy() == y_pred).sum()))
print("Errores:", int((y_test.to_numpy() != y_pred).sum()))
```

```
METRICAS FINALES
accuracy: 0.9722
balanced_accuracy: 0.9762
precision_macro: 0.9744
recall_macro: 0.9762
f1_macro: 0.9743

REPORTE DE CLASIFICACION
              precision    recall  f1-score   support

           0     0.9231    1.0000    0.9600        12
           1     1.0000    0.9286    0.9630        14
           2     1.0000    1.0000    1.0000        10

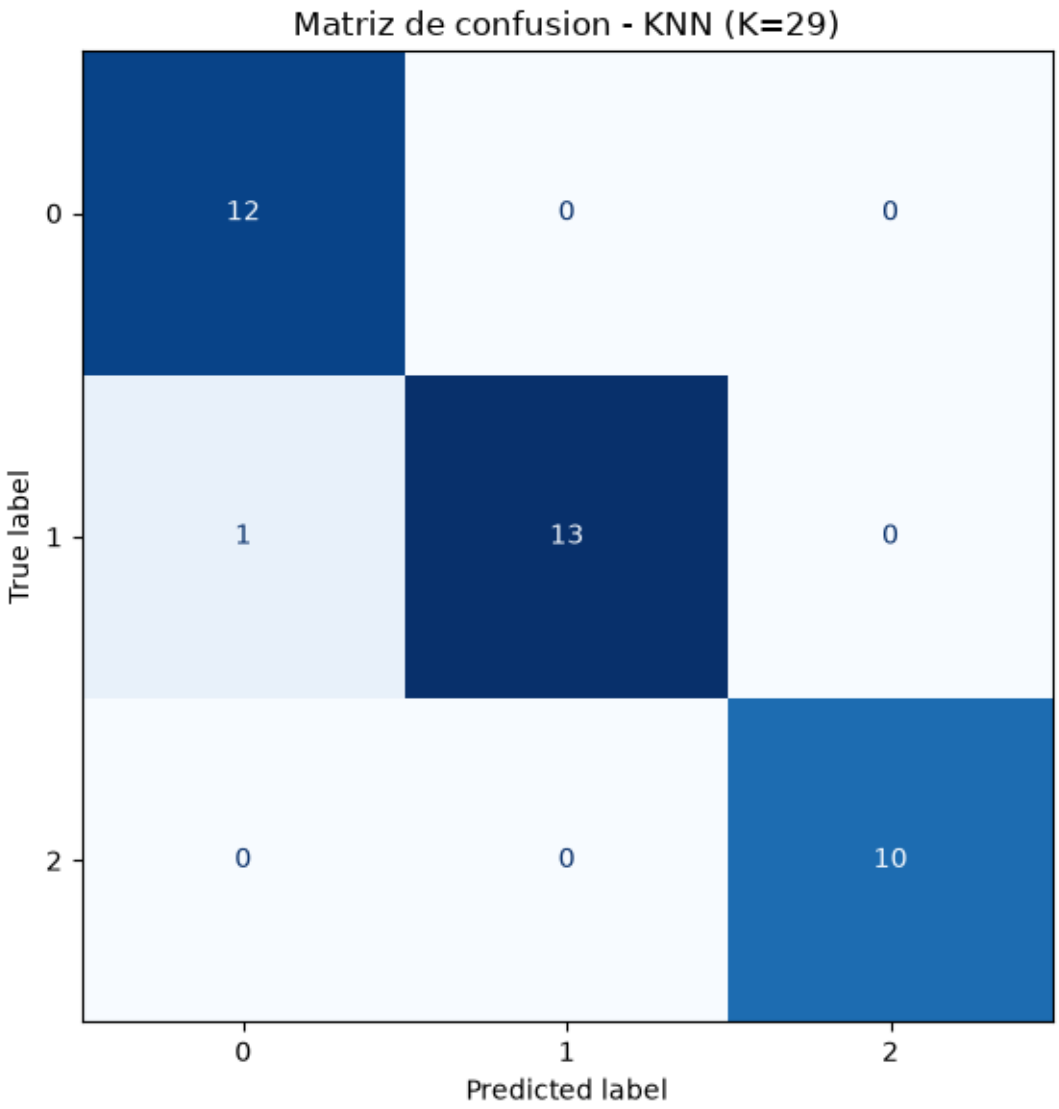
    accuracy                         0.9722        36
   macro avg     0.9744    0.9762    0.9743        36
weighted avg     0.9744    0.9722    0.9723        36

Aciertos: 35
Errores: 1
```

[8]:
```
fig, ax = plt.subplots(figsize=(7, 6))

ConfusionMatrixDisplay.from_predictions(
    y_test,
    y_pred,
    labels=[0, 1, 2],
    cmap="Blues",
    values_format="d",
    ax=ax,
    colorbar=False
)

ax.set_title(f"Matriz de confusion - KNN (K={mejor_k})")
plt.tight_layout()
plt.show()
```



1.6 5. Predicción de una observación nueva

Se construye un ejemplo sintético con 13 características químicas. No se conoce su etiqueta real.

La distribución mostrada corresponde a las fracciones de votos de KNN, no necesariamente a probabilidades calibradas.

[9]:
```
muestra = {
    "alcohol": 13.50,
    "malic_acid": 1.90,
    "ash": 2.50,
    "alcalinity_of_ash": 17.00,
    "magnesium": 105.00,
```

```
    "total_phenols": 2.60,
    "flavanoids": 2.90,
    "nonflavanoid_phenols": 0.29,
    "proanthocyanins": 1.80,
    "color_intensity": 5.40,
    "hue": 1.05,
    "od280/od315_of_diluted_wines": 3.10,
    "proline": 1000.00
}

nuevo = pd.DataFrame([muestra])[X.columns]

clase = modelo_final.predict(nuevo)[0]
votos = modelo_final.predict_proba(nuevo)[0]

print("Clase predicha:", clase)
for etiqueta, fraccion in zip(
    modelo_final.named_steps["knn"].classes_, votos
):
    print(f"Clase {etiqueta}: {fraccion:.2%}")
```

```
Clase predicha: 0
Clase 0: 100.00%
Clase 1: 0.00%
Clase 2: 0.00%
```

1.7 6. Conclusiones

En el experimento original, el mejor valor entre K=1 y K=60 fue **K=29**, con un accuracy promedio de validación cruzada de **97.88%**.

En la prueba final se obtuvieron **35 aciertos de 36**, equivalentes a **97.22% de accuracy**, y un F1 macro aproximado de **97.43%**.

El único error correspondió a una observación de clase 1 clasificada como clase 0.

1.7.1 Limitaciones

- El conjunto de datos tiene solamente 178 observaciones.
- El conjunto de prueba contiene 36 observaciones.
- K=29 es el mejor valor observado dentro del intervalo evaluado.
- Los resultados no garantizan el mismo rendimiento en otros datos.
- La predicción sintética no constituye una evaluación adicional.

1.7.2 Mejoras futuras

Comparar otros algoritmos, ampliar la validación y realizar experimentos con nuevos conjuntos de datos independientes.

Fuente: Wine Recognition Dataset, UCI Machine Learning Repository; disponible mediante `sklearn.datasets.load_wine`.